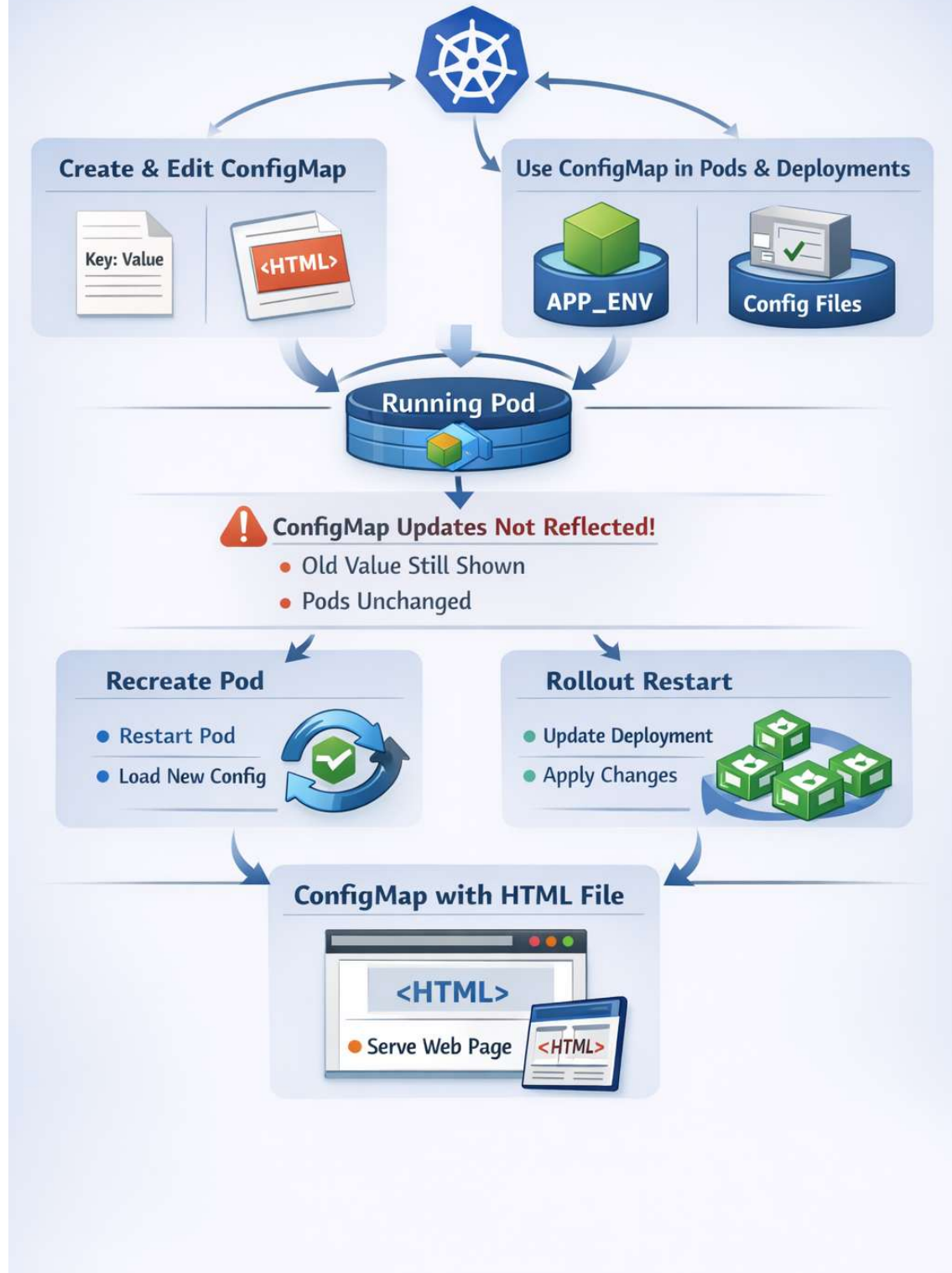


Kubernetes ConfigMap

Implementation & Usage



Kubernetes ConfigMap Implementation

A ConfigMap is an API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume.

A ConfigMap allows you to decouple environment-specific configuration from your container images, so that your applications are easily portable.

ConfigMap object

A ConfigMap is an API object that lets you store configuration for other objects to use. Unlike most Kubernetes objects that have a spec, a ConfigMap has data and binaryData fields. These fields accept key-value pairs as their values. Both the data field and the binaryData are optional. The data field is designed to contain UTF-8 strings while the binaryData field is designed to contain binary data as base64-encoded strings.

The name of a ConfigMap must be a valid DNS subdomain name.

Each key under the data or the binaryData field must consist of alphanumeric characters, -, _ or .. The keys stored in data must not overlap with the keys in the binaryData field.

A ConfigMap is not designed to hold large chunks of data. The data stored in a ConfigMap cannot exceed 1 MiB. If you need to store settings that are larger than this limit, you may want to consider mounting a volume or use a separate database or file service.

ConfigMaps and Pods

You can write a Pod spec that refers to a ConfigMap and configures the container(s) in that Pod based on the data in the ConfigMap. The Pod and the ConfigMap must be in the same namespace

The spec of a static Pod cannot refer to a ConfigMap or any other API objects.

To learn how to manage application configuration in Kubernetes using ConfigMaps and apply it in real scenarios like environment variables and dynamic web content.

Detailed Objective Explanation

- To learn how a Kubernetes ConfigMap is used to store non-confidential configuration data in key-value format.
- To understand how ConfigMaps help in decoupling environment-specific settings from application code, improving portability and flexibility.
- To practice creating, editing, and managing ConfigMaps in Kubernetes.
- To implement ConfigMaps in Pods using:
 - Environment variables (env, envFrom)
 - Configuration files inside containers
- To observe how ConfigMap updates behave with running Pods (i.e., changes are not reflected until Pods are restarted).
- To learn how to apply updated configurations using:
 - Pod recreation
 - Deployment rollout restart
- To understand how ConfigMaps can store not only variables but also files like HTML, scripts, or configs.
- To implement a real use case by:
 - Storing HTML content in a ConfigMap
 - Injecting it into a container
 - Dynamically generating and serving a web page

Create a ConfigMap

```
controlplane:~$ kubectl create configmap my-cm --from-literal=Name=Shiva --from-literal=Country=India --from-literal=State=Maharashtra
configmap/my-cm created
controlplane:~$ kubectl get configmaps
NAME          DATA   AGE
kube-root-ca.crt  1      16d
my-cm          3      12s
controlplane:~$ kubectl describe configmaps my-cm
Name:         my-cm
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
Country:
-----
India

Name:
-----
Shiva

State:
-----
Maharashtra

BinaryData
=====

Events: <none>
controlplane:~$
```

Using Configmaps as environment variables

To use a Configmap in an environment variable in a Pod:

1. For each container in your Pod specification, add an environment variable for each Configmap key that you want to use to the `env[].valueFrom.configMapKeyRef` field.
2. Modify your image and/or command line so that the program looks for values in the specified environment variables.

This is an example of defining a ConfigMap as a pod environment variable:

Edit ConfigMap (Add / Remove ENV)

Example update:

data:

Dist: Chh.Sambhajinagar

```

# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  Country: India
  Name: Shiva
  State: Maharashtra
  Dist: Chh.Sambhajinagar
kind: ConfigMap
metadata:
  creationTimestamp: "2026-02-17T12:26:07Z"
  name: my-cm
  namespace: default
  resourceVersion: "3423"
  uid: 00361708-9c1f-47d6-8782-9d621329a3ff
~
~

```

Check env

```

controlplane:~$ kubectl edit configmaps my-cm
configmap/my-cm edited
controlplane:~$ kubectl describe configmaps my-cm
Name:          my-cm
Namespace:     default
Labels:        <none>
Annotations:   <none>

Data
====
Country:
----
India

Dist:
----
Chh.Sambhajinagar

Name:
----
Shiva

State:
----
Maharashtra

```

Create Pod YAML

File: podcm.yaml

The `envFrom` field instructs Kubernetes to create environment variables from the sources nested within it. The inner `configMapRef` refers to a ConfigMap by its name and selects all its key-value pairs. Add the Pod to your cluster, then retrieve its logs to see the output from the `printenv` command. This should confirm that the two key-value pairs from the ConfigMap have been set as environment variables

Create Pod YAML with ConfigMap Reference

```
controlplane:~$ vi podcm.yaml
controlplane:~$ cat podcm.yaml
apiVersion: v1
kind: Pod
metadata:
  name: podcm
spec:
  containers:
  - name: app
    image: httpd:latest
    ports:
    - containerPort: 80
    envFrom:
    - configMapRef:
      name: my-cm
```

Creates a Pod using the defined YAML file.

```
controlplane:~$ kubectl apply -f podcm.yaml
pod/podcm created
controlplane:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
podcm     1/1     Running   0           15s
controlplane:~$
```

Verify Environment Variables

Checks whether the ConfigMap values are injected into the pod

Displays all environment variables inside the pod

```
controlplane:~$ kubectl exec -it podcm -- bash
root@podcm:/usr/local/apache2# printenv
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
Name=Shiva
HOSTNAME=podcm
PWD=/usr/local/apache2
HTTPD_VERSION=2.4.66
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
HTTPD_PATCHES=
TERM=xterm
HTTPD_SHA256=94d7ff2b42acbb828e870ba29e4cbad48e558a79c623ad3596e4116efcfea25a
Dist=Chh.Sambhajinagar
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
Country=India
KUBERNETES_PORT_443_TCP_PORT=443
State=Maharashtra
PATH=/usr/local/apache2/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HTTPD_PREFIX=/usr/local/apache2
_=/usr/bin/printenv
root@podcm:/usr/local/apache2#
```

Update ConfigMap Value

I have change a configmap env from chh.Sambhajinagar to Yavatmal

```
controlplane:~$ kubectl edit configmaps my-cm
configmap/my-cm edited
controlplane:~$ kubectl describe configmaps my-cm
Name:          my-cm
Namespace:     default
Labels:        <none>
Annotations:   <none>

Data
====
Country:
----
India

Dist:
----
Yavatmal

Name:
----
Shiva

State:
----
Maharashtra

BinaryData
=====
```

Observe Behavior in Existing Pod

Observation:

- Existing Pod **does NOT** update
- Still shows old value

ConfigMap updates do not affect running Pods.

Result:

Existing Pod still shows:

Dist =Chh.Sambhajanagar

ConfigMap updates do not affect running Pods.

```
controlplane:~$ kubectl exec -it podcm -- bash
root@podcm:/usr/local/apache2# printenv
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
Name=Shiva
HOSTNAME=podcm
PWD=/usr/local/apache2
HTTPD_VERSION=2.4.66
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
HTTPD_PATCHES=
TERM=xterm
HTTPD_SHA256=94d7ff2b42acbb828e870ba29e4cbad48e558a79c623ad3596e4116efcfea25a
Dist=Chh.Sambhajanagar
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
Country=India
KUBERNETES_PORT_443_TCP_PORT=443
State=Maharashtra
PATH=/usr/local/apache2/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HTTPD_PREFIX=/usr/local/apache2
_=/usr/bin/printenv
root@podcm:/usr/local/apache2#
```

Recreate Pod

Recreates the Pod to load updated ConfigMap values.

Result:

Dist =Yavatmal


```

controlplane:~$ kubectl delete pod podcm
pod "podcm" deleted from default namespace
controlplane:~$ kubectl get pod -o wide
No resources found in default namespace.
controlplane:~$ kubectl apply -f podcm.yaml
pod/podcm created
controlplane:~$ kubectl exec -it podcm --bash
error: unknown flag: --bash
See 'kubectl exec --help' for usage.
controlplane:~$ kubectl exec -it podcm -- bash
root@podcm:/usr/local/apache2# printenv
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
Name=Shiva
HOSTNAME=podcm
PWD=/usr/local/apache2
HTTPD_VERSION=2.4.66
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
HTTPD_PATCHES=
TERM=xterm
HTTPD_SHA256=94d7ff2b42acbb828e870ba29e4cbad48e558a79c623ad3596e4116efcfea25a
Dist=Yavatmal
SHLV=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
Country=India
KUBERNETES_PORT_443_TCP_PORT=443

```

Create Deployment YAML

File: dep-cm.yaml

```

controlplane:~$ kubectl create deployment dep-cm --image=docker.io/httpd --dry-run=client -o yaml > dep-cm.yaml
controlplane:~$ vi dep-cm.yaml

```

Edit Deployment YAML to Attach ConfigMap

```

apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: dep-cm
    name: dep-cm
spec:
  replicas: 1
  selector:
    matchLabels:
      app: dep-cm
  strategy: {}
  template:
    metadata:
      labels:
        app: dep-cm
    spec:
      containers:
        - image: docker.io/httpd
          name: httpd
          resources: {}
          envFrom:
            - configMapRef:
                name: my-cm
status: {}
~
~

```

Create a Deployment from Pod yaml file

```
controlplane:~$ kubectl apply -f dep-cm.yaml
deployment.apps/dep-cm created
controlplane:~$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE   NOMINATED NODE   READINESS GATES
dep-cm-74cddb7cf-1759g             1/1     Running   0           15s   192.168.1.7    node01   <none>           <none>
podcm                               1/1     Running   0           16m   192.168.1.6    node01   <none>           <none>
controlplane:~$
```

Check ENV in Deployment Pod

ENV loaded successfully

```
controlplane:~$ kubectl exec -it dep-cm-74cddb7cf-1759g -- bash
root@dep-cm-74cddb7cf-1759g:/usr/local/apache2# printenv
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
Name=Shiva
HOSTNAME=dep-cm-74cddb7cf-1759g
PWD=/usr/local/apache2
HTTPD_VERSION=2.4.66
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
HTTPD_PATCHES=
TERM=xterm
HTTPD_SHA256=94d7ff2b42acbb828e870ba29e4cbad48e558a79c623ad3596e4116efcfea25a
Dist=Yavatmal
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
Country=India
KUBERNETES_PORT_443_TCP_PORT=443
State=Maharashtra
PATH=/usr/local/apache2/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HTTPD_PREFIX=/usr/local/apache2
_=/usr/bin/printenv
```

Edit ConfigMap (Add New ENV)

kubectl edit configmap my-cm

Add:

sname: Bedre

```

Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  Country: India
  Dist: Yavatmal
  Name: Shiva
  Sname: Bedre
  State: Maharashtra
kind: ConfigMap
metadata:
  creationTimestamp: "2026-02-17T13:51:01Z"
  name: my-cm
  namespace: default
  resourceVersion: "6280"
  uid: bc103d0c-2518-4bd6-8651-a6eaed80d0bd
~

```

Check Pod (No Update)

New variable not visible , because need to restart or recreate a pod

```

controlplane:~$ kubectl edit configmaps my-cm
configmap/my-cm edited
controlplane:~$ kubectl exec -it dep-cm-74cddb7cf-1759g -- bash
root@dep-cm-74cddb7cf-1759g:/usr/local/apache2# printenv
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
Name=Shiva
HOSTNAME=dep-cm-74cddb7cf-1759g
PWD=/usr/local/apache2
HTTPD_VERSION=2.4.66
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
HTTPD_PATCHES=
TERM=xterm
HTTPD_SHA256=94d7ff2b42acbb828e870ba29e4cbad48e558a79c623ad3596e4116efcfea25a
Dist=Yavatmal
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
Country=India
KUBERNETES_PORT_443_TCP_PORT=443
State=Maharashtra
PATH=/usr/local/apache2/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HTTPD_PREFIX=/usr/local/apache2
_=/usr/bin/printenv
root@dep-cm-74cddb7cf-1759g:/usr/local/apache2#

```

Restart Deployment

Rollout restart safely refreshes pods to apply new configuration without downtime.

What it does:

- Terminates old pods one by one
- Creates new pods automatically
- Applies updated configuration (like ConfigMap changes)

Why we use it:

- To apply updated ConfigMap or Secret changes
- To avoid manual pod deletion
- To ensure zero downtime deployment

Verify Updated ENV

Output:

sname=Bedre

```
controlplane:~$ kubectl rollout restart deployment dep-cm
deployment.apps/dep-cm restarted
controlplane:~$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE   NOMINATED NODE   READINESS GATES
dep-cm-5dc79c7b9c-gnxxl            1/1     Running   0           15s   192.168.1.8     node01   <none>            <none>
podcm                               1/1     Running   0           29m   192.168.1.6     node01   <none>            <none>
controlplane:~$ kubectl exec -it dep-cm-5dc79c7b9c-gnxxl -- bash
root@dep-cm-5dc79c7b9c-gnxxl:/usr/local/apache2# printenv
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
Name=Shiva
HOSTNAME=dep-cm-5dc79c7b9c-gnxxl
PWD=/usr/local/apache2
HTTPD_VERSION=2.4.66
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
HTTPD_PATCHES=
TERM=xterm
HTTPD_SHA256=94d7ff2b42acbb828e870ba29e4cbad48e558a79c623ad3596e4116efcfea25a
Dist=Yavatmal
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
sname=Bedre
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
Country=India
KUBERNETES_PORT_443_TCP_PORT=443
State=Maharashtra
PATH=/usr/local/apache2/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HTTPD_PREFIX=/usr/local/apache2
_/usr/bin/printenv
```

Web Content Section

Create Web Content File

```
controlplane:~$ cat cmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: httpd-html-config
data:
  INDEX_CONTENT: |
    <!DOCTYPE html>
    <html>
    <body>
      <h1>Unnati KUCL </h1>
      <p>Apache HTTPD server running with ConfigMap data.</p>
    </body>
    </html>
```

Create ConfigMap from Web Content File

Kubectl apply -f cmap.yaml

What i did: Stored HTML file inside a ConfigMap

Concept:

ConfigMap is not only for ENV variables It can also store files (like HTML, configs, scripts)

Task Purpose:

- Convert file → Kubernetes configuration
- Make it usable inside containers

Add Web Content ConfigMap in Deployment YAML

- Read data from ConfigMap
- Store it in an environment variable (HTML_DATA)
- Use a command to write that data into an HTML file
- Start the web server

```

controlplane:~$ cat dep-cm.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: dep-cm
    name: dep-cm
spec:
  replicas: 1
  selector:
    matchLabels:
      app: dep-cm
  strategy: {}
  template:
    metadata:
      labels:
        app: dep-cm
    spec:
      containers:
      - image: docker.io/httpd
        name: httpd
        resources: {}
        env:
        - name: HTML_DATA
          valueFrom:
            configMapKeyRef:
              name: httpd-html-config
              key: INDEX_CONTENT
          command: ["/bin/sh", "-c"]
          args:
            - |
              echo "$HTML_DATA" > /usr/local/apache2/htdocs/index.html &&
              httpd-foreground
        ports:
        - containerPort: 80

status: {}
controlplane:~$

```

- Took HTML content from ConfigMap
- Converted it into a file inside container
- Served it using Apache (httpd)

Converted ConfigMap data into a runtime-generated HTML file using environment variables and shell command.

Verify

```

controlplane:~$ curl 192.168.1.9
<!DOCTYPE html>
<html>
<body>
  <h1>Unnati KUCL </h1>
  <p>Apache HTTPD server running with ConfigMap data.</p>
</body>
</html>

```

Stored HTML in ConfigMap and dynamically generated a webpage inside container using environment variables.

Conclusion

This task demonstrated how Kubernetes ConfigMaps are used to manage application configuration separately from container images. It showed how to apply ConfigMaps in Pods and Deployments, and highlighted that changes require restarting or recreating Pods to take effect. Overall, it helps in building flexible and easily manageable applications.